Tampereen yliopisto

COMP.SE.110
SOFTWARE DESIGN
MID-TERM DOCUMENTATION
Event-Planner Dashboard: Weather and Air Quality

Naveed Ali
Affira
Zhiqing Feng
Janika Ranta

Table of Contents

1. **PROJECT OVERVIEW** 3
2. **STRUCTURE AND KEY COMPONENTS** 3
3. **COMPONENTS AND INTERFACES** 3
4. **INTERNAL COMPONENT DETAILS** 5
5. **DESIGN PATTERNS AND ARCHITECTURE** 7
6. **APIS AND DATA** 11
7. **SAVED PREFERENCES** 12
8. **HISTORY** 13
9. **USE OF AI** 13
10. **TENTATIVE APPROACHES** 14
11. **SELF-EVALUATION** 14
12. **FUTURE WORK** 16
13. **CONCLUSION** 17
14. **DIVISION OF WORK** 18

Event Planner's Dashboard

1. Project Overview

The **Event Planner's Dashboard** is designed to help event planners choose the best dates and locations for outdoor events based on real-time weather forecasts and air quality data. The application fetches data from two APIs: one for weather information and another for air quality metrics, allowing users to make informed decisions for event scheduling. The dashboard provides customizable visualizations for weather conditions and air quality levels. Users can save their preferences for future use and access their historical data to compare trends over time.

User Stories/Use Cases:

- **As a professional event planner**, I want to view detailed weather and air quality forecasts to choose the best dates for outdoor events.
- **As an event venue manager**, I want to compare historical data with current weather trends to make informed decisions about venue bookings.
- **As a casual user**, I want to check real-time weather and air quality data for my favorite locations so I can plan outings and activities.

2. Structure and Key Components

The dashboard is divided into three main sections:

1. **Weather Data Section**: Displays data such as temperature, humidity, and wind speed in line graphs.
2. **Air Quality Data Section**: Visualizes air quality metrics (e.g., PM2.5, PM10) using bar charts.
3. **The Map and Preferences Section** allows users to select locations from a map and save their preferences for easy future access.

Main Components:

- **Location and Date Picker**: A dropdown menu for selecting regions and a date picker for specifying date ranges.
- **Weather Visualizations** represent crucial weather metrics such as temperature, humidity, and wind speed. The data is fetched from an external API and displayed in interactive line charts for better visual interpretation.
- **Air Quality Visualizations**: This service uses bar charts to visualize PM2.5 and PM10 levels. These indicators help users understand air quality trends and how they might affect events.
- **The Map and Preferences Section** allows users to select locations on a map and save them for easy future access. Preferences include the date range and data filters for weather and air quality metrics.

3. Components and Interfaces

The project will utilize a **Model-View-Controller (MVC)** architecture to separate data processing, user interaction, and interface rendering. Here are the core components:

1. **Weather API Handler**:

   It handles requests to external weather data providers (e.g., OpenWeatherMap or the Finnish Meteorological Institute). It fetches weather details such as

temperature, humidity, wind speed, and precipitation levels in JSON format. The handler validates responses and parses them into internal data models.

2. **Air Quality API Handler**:
   It connects to the OpenAQ API to fetch real-time air quality data. It retrieves JSON responses containing metrics like PM2.5 and PM10. The handler is responsible for converting this data into a format usable by the visualization components.
3. **Controller**:
   The controller receives user input from the UI, triggers the appropriate API handlers, and passes the fetched data to be displayed. It coordinates all interactions between the API handlers and the UI components.
4. **User Interface (UI)**:
   The UI, built using JavaFX, displays real-time data visualizations. It allows users to input locations, select dates, save preferences, and view historical weather conditions. The UI components include line graphs for weather data, bar charts for air quality, and a map-based location picker.

Class Diagram:

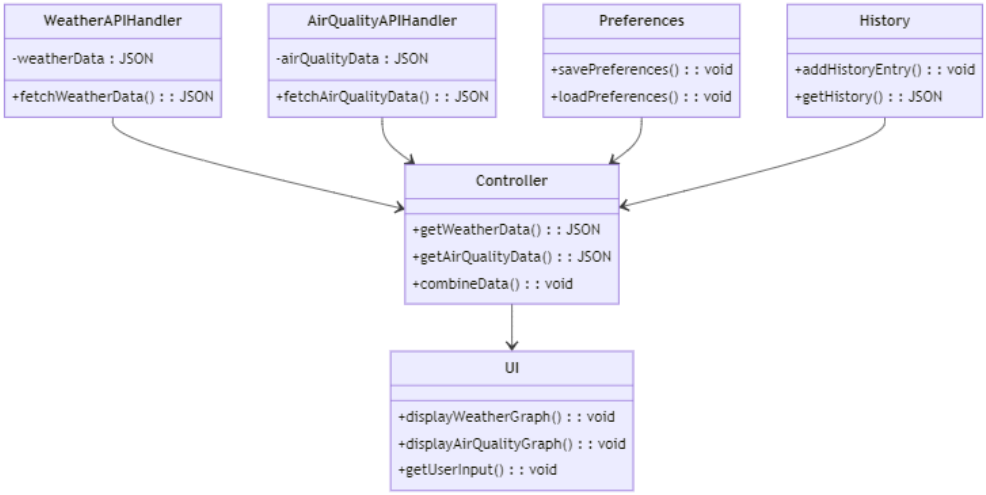


Figure 1 Event Planner's Dashboard Class Diagram

4. Internal Component Details

1. Controller Layer

- Base Controller

Core Navigation Component

- Manages common navigation logic across all views
- Implements scene caching mechanism for improved performance
- Handles basic UI controls (home, history, preferences, about buttons)
- Uses FXML injection for UI components

- Dashboard Controller

Main Application Controller

- Extends BaseController for consistent navigation
- Manages multiple UI Components:
  - Weather visualization (LineChart)
  - Air quality visualization (BarChart)
  - Interactive calendar with weather highlighting
  - Search functionality
  - Weather summary display
- Implements comprehensive calendar management:
  - Monthly view navigation
  - Weather condition highlighting
  - Best/worst day identification
- Handles data visualization:
  - Temperature and humidity line charts
  - Air quality bar charts
  - Weather summary updates

2. Service Layer

- WeatherApiService

Weather Data Management

- Connects to OpenWeatherMap API
- Implements 5-day weather forecast fetching
- Features:
  - City-based weather data retrieval
  - Temperature, humidity, and precipitation data processing
  - Data aggregation for daily summaries
  - Error handling and response validation
- Data processing
  - Converts raw JSON responses to WeatherData objects
  - Calculates daily averages and extremes
  - Handles multiple weather metrics (temperature, humidity, precipitation)

- OpenAQService

Air Quality Data Management

- Integrates with OpenAQ Platform API
- Features:
  - City-based air quality data retrieval
  - Multiple pollutant measurement tracking (PM2.5, PM10, NO2)
  - Robust error handling
- Data Processing
  - Parses complex JSON responses
  - Extracts relevant air quality parameters
  - Provides normalized data structure through the AirQualityData model

3. Data Flow Architecture

- User Input Processing

User Input → DashboardController → API Services → Data Models → UI Updates

- Data Visualization Flow

API Response → Data Models → Controller Processing → Chart Updates

- Calendar Integration

Weather Data → Date Processing → Calendar Highlighting → UI Updates

4. Component Interactions

- Controller-Service Integration

DashboardController orchestrates between services:

- Coordinates weather and air quality data fetching
- Manages concurrent data updates
- Handles service response integration

- Data Synchronization
  - Implements real-time data updates
  - Manages state between different views
  - Coordinates between calendar and weather data
- Error Handling Strategy
  - Centralized error management
  - User-friendly error messages
  - Graceful fallback mechanisms

5. Technical Implementation Details

- Caching Mechanism

EPDWA.getInstance().getLoadedScenes().containsKey(fxml)

  - Implements scene caching for improved performance
  - Reduces memory usage
  - Maintains UI state between navigation

- Data Aggregation

Map<String, List<WeatherData>> dailyDataMap = new HashMap<>();

  - Efficient data grouping
  - Optimized for daily weather summaries
  - Reduces API calls

1. **UI Update Patterns**

Platform.runLater(() -> updateUI(weatherDataModel, airQualityData));

  - Thread-safe UI updates
  - Asynchronous data loading
  - Responsive user interface

5. Design Patterns and Architecture

Several design patterns and architectural principles are employed for scalability, maintainability, and efficiency:

1. Model-View-Controller (MVC):

- Implementation details:

```
// Model Layer
public class DataModel {
private List<WeatherData> weatherDataList;
// Handles data representation
}
```

```
// View Layer (FXML)
// Dashboard.fxml, History.fxml,
SavePreferences.fxml, About.fxml

// Controller Layer
public class DashboardController extends
BaseController {
// Handles user interactions and view updates
}
```

- Components:
  - **Models**: DataModel, WeatherData, AirQualityData
  - **Views**: FXML files for layout definition
  - **Controllers**: Hierarchy of controllers managing user interaction
- Benefits:
  - Clear separation of UI, business logic, and data
  - Independent testing of components
  - Maintainable codebase structure

2. Static Registry Pattern

```
public class ControllerHolder {
    private static HistoryController
historyController;
    private static DashboardController
dashboardController;
    // ...

    public static void
setHistoryController(HistoryController controller) {
        historyController = controller;
    }

    public static HistoryController
getHistoryController() {
        return historyController;
    }
}
```

- Purpose:
  - Centralized controller management
  - Global access to controller instances
  - Communication between controllers
- Benefits:
  - Simplified controller access
  - Consistent state across views
  - Reduced coupling between controllers

3. Template Method Pattern (BaseController)

```
public class BaseController {
    @FXML protected Button homeButton;
    // ... other buttons

    @FXML
    protected void initialize() {
        setupNavigation();
    }

    protected void navigateTo(String fxml) {
        // Common navigation logic
    }
}

public class DashboardController extends
BaseController {
    @FXML
    @Override
    protected void initialize() {
        super.initialize();
        // Additional initialization
    }
}
```

- Implementation:
  - Base navigation and initialization logic in BaseController
  - Specialized behavior in child controllers
  - Common UI element handling
- Advantages:
  - Reusable navigation logic
  - Consistent behavior across views
  - Reduced code duplication

4. Service Layer Pattern

```
public class WeatherApiService {
    public DataModel fetchWeatherData(String
cityName) {
        // API interaction logic
    }
}

public class OpenAQService {
```

```
    public AirQualityData fetchAirQualityData(String
cityName) {
        // API interaction logic
    }
}
```

- Features:
  - Encapsulated API communication
  - Separate services for different data types
  - Clean interface for controllers
- Benefits:
  - Isolated external service interaction
  - Simplified error handling
  - Easy maintenance and updates

5. Scene Caching Pattern

```
// In BaseController
protected void navigateTo(String fxml) {
    EPDWA epdwa = EPDWA.getInstance();
    if (epdwa.getLoadedScenes().containsKey(fxml)) {
        Scene existingScene =
epdwa.getLoadedScenes().get(fxml);
        // Use cached scene
    } else {
        // Load and cache new scene
    }
}
```

- Implementation:
  - Scene caching in EPDWA singleton
  - Lazy loading of views
  - Memory-efficient navigation
- Advantages:
  - Improved performance
  - Reduced memory usage
  - Smooth navigation experience

6. Observer Pattern (Event Handling)

- Implementation:
  - JavaFX event handling system
  - Property binding for real-time updates
  - Scene graph updates

- Use Cases:
  - Weather data updates
  - UI refresh on data changes
  - Calendar view updates

This architecture combines multiple design patterns to create a robust, maintainable, efficient application structure. Each pattern addresses specific concerns while working together to provide a cohesive solution.

6. APIs and Data

The application will retrieve data from two main APIs:

1. Weather API:

- Finnish Meteorological Institute's Open Data API and Open Weather's API provide real-time weather data, such as temperature, humidity, wind speed, and precipitation forecasts.
- **Example Endpoint**: GET https://opendata.fmi.fi/weather/forecast?location=
- **Data Type**: JSON response with forecast details over a selected date range.

2. Air Quality API:

- The Open AQ Platform API provides real-time air quality data, including PM2.5 and PM10 levels.
- **Example Endpoint**: GET https://api.openaq.org/v1/latest?coordinates=
- **Data Type**: JSON response with air quality metrics for the selected location.

Integration with APIs

The Event Planner's Dashboard interacts with the OpenWeatherMap API to fetch and present real-time weather data. The **WeatherApiService** class manages this integration, which includes the following steps:

1. **Data Retrieval**
   The service sends a GET request to the OpenWeatherMap API using a URL constructed with the user's city name and API key. The URL is prepared by encoding the city name and attaching additional query parameters, such as units (metric system), to ensure consistency in temperature values.
2. **Response Validation and Error Handling**
   The service checks the response code returned by the API. If an error code (e.g., 404 for city not found or 401 for an invalid API key) is encountered, the service throws an appropriate error message to notify the user. This approach ensures that invalid requests do not cause unexpected application behavior.
3. **Data Parsing and Processing**
   The API's JSON response is parsed to extract critical weather information, such as temperature, humidity, precipitation levels, and a brief description. The service then constructs a **WeatherData** object to encapsulate this information.
4. **Controller Integration**
   The parsed weather data is returned to the controller as a DataModel object, which updates the application's user interface. This separation of concerns allows the

controller to focus on managing UI updates while the service handles data fetching and parsing.

5. **Data Retrieval and Parsing Example**:

```
public DataModel fetchWeatherData(String cityName) {
    // Clean and encode the city name for the API request
    String encodedCity = URLEncoder.encode(cityName.trim(),
StandardCharsets.UTF_8);
    String apiUrl = String.format("%s?q=%s&units=metric&appid=%s",
BASE_URL, encodedCity, API_KEY);

    // Send the GET request and parse the JSON response
    try {
        String jsonResponse = sendGetRequest(apiUrl);
        WeatherData weatherData = parseWeatherData(jsonResponse);
        return new DataModel(List.of(weatherData));
    } catch (Exception e) {
        // Handle errors and return an empty model if needed
        System.err.println("Error fetching weather data: " +
e.getMessage());
        return new DataModel(new ArrayList<>());
    }
}

private String sendGetRequest(String urlString) throws IOException {
    HttpURLConnection conn = (HttpURLConnection) new
URL(urlString).openConnection();
    conn.setRequestMethod("GET");
    conn.setRequestProperty("Accept", "application/json");

    // Validate response code and return the response as a string
    if (conn.getResponseCode() == HttpURLConnection.HTTP_OK) {
        try (BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()))) {
            return reader.lines().collect(Collectors.joining());
        }
    } else {
        throw new RuntimeException("Failed to fetch data: HTTP error
code " + conn.getResponseCode());
    }
}
```

This structured API integration approach ensures reliable data retrieval, graceful error handling, and seamless user experience.

7. Saved Preferences

The **Saved Preferences** feature allows users to store frequently used configurations, making it easy to reload previously selected locations, date ranges, and metrics without repeating the setup process.

Key Features:

1. **Preference Name**: Users can give a custom name to each saved configuration.
2. **Location and Date Range**: Saves the chosen region and date range for repeated queries.

3. **Saved Metrics**: Stores preferred weather and air quality metrics such as temperature, PM2.5, humidity, etc.
4. **Preference Actions:** Users can load, edit, or delete saved preferences through an intuitive UI.
5. **Data Storage Strategy:** Preferences are serialized into a JSON file, enabling easy retrieval and updates.
6. **Quick Overview (Optional)**: Displays a mini-summary or chart preview of the saved data.

Screen Layout:

- **Table View**: A simple table with columns for preference name, location, date range, and saved metrics.
- **Card Layout (Optional)**: Cards showing saved configurations with action buttons (Load, Edit, Delete).

8. History

The **History** section allows users to revisit previous queries, making it easy to compare past weather and air quality conditions with current data.

Key Features:

1. **Timestamp:** Records when the user last accessed a specific location or preference.
2. **Location and Date Range**: Displays previously selected regions and date ranges.
3. **Weather and Air Quality Summary**: Provides a snapshot of the weather and air quality conditions at the time.
4. **Actions**: Users can view past data or compare multiple historical entries.
5. **Optional Data**: Icons and short summaries of the past conditions for quick reference.

Screen Layout:

- **Timeline View**: A vertical timeline of past entries with timestamps, summaries, and action buttons (View Again, Compare).
- **Table View (Alternative)**: A standard table with columns for timestamp, location, weather, and air quality.

9. Use of AI

1. Tools Used

- ChatGPT
  - Used for code review and optimization suggestions
  - Helped in debugging complex issues
  - Assisted in generating boilerplate code
- GitHub Copilot
  - Used for real-time code suggestions
  - Helped with API integration code
  - Assisted in writing unit tests

2. Approach to AI Usage

- Prompt Engineering
  - Structured prompts to get specific architectural suggestions
  - Used AI to review and suggest improvements to existing code
  - Generated test cases based on requirements

3. Effectiveness Analysis

- Advantages
  - Accelerated development process
  - Improved code quality through suggestions
  - Helped identify potential issues early
  - Provided alternative implementation approaches
- Disadvantages
  - Sometimes generated overly complex solutions
  - Required careful validation of suggestions
  - Occasional misunderstanding of context
  - Dependencies on external services

10. Tentative Approaches

In the prototype phase, the following approaches are proposed:

1. **Data Retrieval**: Asynchronous data fetching using Java's Completable Future ensures a responsive user experience by performing API calls without blocking the main thread.
2. **Visualization**: Simple line and bar graphs for weather and air quality visualizations will be used. Technologies like JavaFX or web-based libraries such as Chart.js will be explored.
3. **Customizability**: The application enables users to set custom filters, save preferences, and adjust parameters like location and date range. User preferences are stored in a local database for fast retrieval.
4. **AI Integration (Future)**: In future versions, AI-based recommendations may be implemented using historical trends to suggest optimal event dates. This could involve applying machine learning models, such as time series forecasting (e.g., ARIMA or LSTM), to identify patterns.

11. Self-Evaluation

In this section, we evaluate the effectiveness of our initial design and how well it has supported the development process. We follow our initial design plan, assess potential areas for improvement, and identify any adjustments needed for implementation.

1. **Successes:**
   - Front-End Design: We applied JavaFX for the front-end design, using SceneBuilder to streamline the creation of view components. This tool enables us to visually design the initial interface and generate corresponding FXML files, which

effectively affects the development process. As a result, we successfully created four key views:

1. About view: Provides users with an overview of the application's purpose and features.
2. Dashboard view: This view provides users with the application's primary interface, displaying real-time weather data, a calendar, and other relevant information.
3. History view: Allows users to view the calendar of plan schedules.
4. Save Preferences view: Enables users to configure their settings.

- IDE choice: We utilized Apache NetBeans to develop and deploy the application.

2. **Challenges:**

Challenge 1: FXML and Controller Synchronization

- **Problem Description:**
  One of the challenges encountered during development was ensuring that **FXML elements were correctly synchronized with their corresponding controller fields and event handlers** in JavaFX. Mismatches between FXML-defined elements and controller fields could lead to runtime errors and unresponsive components.
- **Planned Solution:**
  To address this issue, we plan to implement automated testing using **JUnit** and **TestFX** to verify that all FXML elements are correctly linked to their controller fields and event handlers. These tests will help ensure that updates made in SceneBuilder or FXML files do not cause unintended discrepancies in the controllers.
- **Planned Testing Approach:**
  1. **Automated FXML Binding Verification:**
     We plan to create automated tests that load each FXML file and check that all annotated fields in the controller are correctly initialized. This will help detect any inconsistencies between the FXML file and the controller class.
  2. **Event Handler Testing:**
     Additional test cases will be designed to verify that FXML elements, such as buttons, are linked to their respective event handlers in the controller. This will prevent accidental misconfigurations during development.
- **Future Steps:**
  1. Write JUnit tests that load each FXML file and confirm that all fields are correctly initialized.
  2. Use TestFX to automate interaction tests for verifying that button clicks trigger the correct methods in the controller.

Challenge 2: Data Visualization Enhancements

- **Problem Description:**
  Another challenge was enhancing the **data visualizations** to be more interactive and user-friendly. The basic charts needed features like tooltips, data filtering, and dynamic updates, which made it difficult for users to explore and interpret the information displayed.
- **Planned Solution:**
  To make the data visualizations more interactive and intuitive, we have outlined several planned enhancements:

- **Adding Interactive Tooltips:**
  We plan to implement tooltips for the weather charts, providing detailed information when users hover over specific data points. This will allow users to view temperature and humidity values quickly and accurately.
  1. **Planned Approach:**
     Utilize JavaFX's built-in Tooltip class to associate tooltips with each data point. Tooltips will dynamically display detailed information based on the hovered data point.
- **Checkbox-Based Data Filtering:**
  To allow users to customize their view, we will add checkboxes that enable them to filter weather data based on specific metrics like temperature and humidity.
  1. **Planned Approach:**
     Modify the updateChart method to check the state of each checkbox and selectively display data series in the chart based on which options are selected.
- **Accessibility and Interactivity Improvements:**
  We plan to introduce accessibility features to improve usability for all users, including those with disabilities. This may include adding screen reader support and enabling keyboard navigation for data exploration.

- **Future Steps:**
  - Develop a prototype of tooltips for data points in the weather chart using JavaFX's Tooltip class.
  - Implement checkbox-based filtering to allow users to control the visibility of different data series.
  - Explore options for accessibility improvements in the charts and labels.

3. **Implementation Progress:**
   - The major views (About, Dashboard, History and SavePreferences) have been implemented according to plan, aligning with the initial structure.
   - The initial data visualizations are functional and provide essential weather and air quality data, but we are still considering further enhancements.

12. Future Work

1. Planned Enhancements

- Advanced Weather Analytics

- Implementation of machine learning for weather pattern prediction
- Addition of severe weather alerts
- Integration with more weather data sources

- UI/UX Improvements

  - Addition of customizable dashboards
  - Enhanced data visualization options
  - Mobile-responsive design implementation
  - Accessibility improvements

- Performance Optimizations
  - Implementation of more efficient data caching
  - Optimization of API calls
  - Improved loading times for historical data

2. Technical Debt Resolution

- Code Quality
  - Implement comprehensive unit test coverage
  - Refactor complex components
  - Improve error handling
- Architecture Improvements
  - Implement microservices architecture for better scalability
  - Add support for offline mode
  - Improve data synchronization

3. New Features

- Social Integration
  - Sharing capabilities for weather reports
  - Collaborative event planning features
  - Community-driven weather reporting
- Extended Functionality
  - Multiple location comparison
  - Advanced filtering options
  - Custom report generation
  - Integration with calendar applications
- Data Analysis
  - Historical trend analysis
  - Weather impact predictions
  - Custom alerts based on user criteria

13. Conclusion

The **Event Planner's Dashboard** is structured around providing event planners with accurate and useful data to make informed decisions about scheduling outdoor events. The separation of core components, clear user interface, and use of design patterns ensure that the application will be scalable and maintainable for future extensions. The addition of **Saved Preferences** and **History** features enhances the user experience by offering convenient data retrieval and comparison functionality.

14. Division of work

The development of the Event Planner's Dashboard involved collaborative efforts across both the front end and backend, with each team member focusing on their assigned areas:

- **Affira:**
  - Took on a leadership role in overseeing the project
  - Primarily focused on backend development:
    - Implemented core functionalities
    - Managed integration between the backend and frontend
    - Developed base controller architecture
  - Key role in debugging and maintaining component connections
  - Implemented navigation and scene caching system
- **Janika:**
  - Significant backend development contributions:
    - Implemented Weather API integration
    - Set up API service and key configurations
    - Ensured efficient weather data fetching and processing
  - Recent contributions:
    - Final merge of the main branch
    - Integration of complete project components
- **Zhiqing Feng:**
  - Led frontend development efforts:
    - Designed UI pages using JavaFX and SceneBuilder
    - Created and styled main views (Dashboard, History, Preferences)
    - Ensured UI consistency
  - Recent contributions:
    - Cleanup and optimization:
      - Removed redundant weather_dashboard folder
      - Cleaned up unnecessary directories
      - Optimized project structure
    - Initial project setup and file organization
    - Implemented History and Save Preferences FXML
- **Naveed Ali:**
  - Collaborated on frontend development:
    - Implemented visual components
    - Created interactive elements
    - Worked on frontend screens alongside Zhiqing
  - Documentation contributions:
    - Organized project structure
    - Provided a clear overview of application goals
    - Documented design approach

Active participation from all team members, with a recent focus on cleanup, optimization, and final integration of components. The team maintained a balanced distribution of responsibilities between frontend and backend development, with clear ownership of specific components while collaborating effectively on shared objectives.